DBSCAN

Philip Loewen, Arusan Surendiran

DBSCAN Algorithm

This section defines the manual implementation of the DBSCAN algorithm. It utilizes a queue in data structure and a pairwise distance matrix to handle cluster expansion efficiently in R.

https://www.geeksforgeeks.org/dsa/basic-operations-for-queue-in-data-structure/

```r
manual_dbscan <- function(data, eps, minPts) {
  n <- nrow(data)
  cluster_assignment <- rep(0, n)  # 0 = unvisited, -1 = noise
  cluster_id <- 0

  # Compute pairwise Euclidean distances to define epsilon-neighborhoods
  dist_mat <- as.matrix(dist(data, method = "euclidean"))

  for (i in 1:n) {
    # Skip point if it has already been visited
    if (cluster_assignment[i] != 0) next

    # Retrieve the epsilon-neighborhood for point i
    neighbors <- which(dist_mat[i, ] <= eps)

    # Check core point condition
    if (length(neighbors) < minPts) {
      cluster_assignment[i] <- -1 # Temporarily classify as a noise point
    } else {
      # Core point verified: create a new density-based cluster
      cluster_id <- cluster_id + 1
      cluster_assignment[i] <- cluster_id

      # Identify directly density-reachable points to start the chain
      seeds <- setdiff(neighbors, i)
```

```r
    # Initialize a queue to establish density-connected points
    queue <- integer(n)
    queue_len <- length(seeds)

    if (queue_len > 0) {
      queue[1:queue_len] <- seeds
    }

    head <- 1
    tail <- queue_len

    # Expand the cluster to satisfy the Maximality requirement
    while (head <= tail) {
      current_p <- queue[head]
      head <- head + 1

      # If previously classified as noise, it is a border point
      if (cluster_assignment[current_p] == -1) {
        cluster_assignment[current_p] <- cluster_id
      }

      # If unvisited, process for density-reachability
      if (cluster_assignment[current_p] == 0) {
        cluster_assignment[current_p] <- cluster_id

        # Retrieve the epsilon-neighborhood of the current point
        new_neighbors <- which(dist_mat[current_p, ] <= eps)

        # If this point is also a core point, expand the reachable chain
        if (length(new_neighbors) >= minPts) {
          # Extract only unvisited or noise points
          valid_new <- new_neighbors[cluster_assignment[new_neighbors] <= 0]
          num_new <- length(valid_new)

          if (num_new > 0) {
            queue[(tail + 1):(tail + num_new)] <- valid_new
            tail <- tail + num_new
          }
        }
      }
    }
  }
```

```r
  }
  return(cluster_assignment)
}
```

Datasets

This section handles the retrieval and preprocessing of the real-world datasets from the UCI Machine Learning Repository.

```r
library(remotes)
#remotes::install_github("coatless-rpkg/ucimlrepo")
library(ucimlrepo)
library(here)
```

```
here() starts at /Users/arusansurendiran/Home/My GitHub/K-DBSCAN
```

```r
# Abalone Dataset
abalone_uci <- fetch_ucirepo(name = "abalone")
abalone <- abalone_uci$data$original
colnames(abalone) <- c("Sex", "Length", "Diameter", "Height", "Whole_w",
                       "Shucked_w", "Viscera_w", "Shell_w", "Rings")

# Prepare and scale Abalone for clustering
abalone_numeric <- abalone[, 2:8]
abalone_scaled <- scale(abalone_numeric)

# MAGIC Gamma Telescope Dataset
MAGIC_uci <- fetch_ucirepo(id=159)
magic <- MAGIC_uci$data$original


# Shuttle Data

shuttle_train <- read.table("data/shuttle.trn", header = FALSE, sep = " ")
shuttle_test <- read.table("data/shuttle.tst", header = FALSE, sep = " ")
statlog_shuttle <- rbind(shuttle_train, shuttle_test)

# Separate features from labels, the last column (V10) is the class label
statlog_features <- statlog_shuttle[, 1:9]
```

Simulation

The simulation evaluates the algorithm's performance on non-linear geometries (sine curves, rings, and arcs) and its ability to distinguish clusters from background noise. We demonstrate the advantages of DBSCAN clustering in comparison to k-means clustering.

```
library(dbscan)
```

```
Attaching package: 'dbscan'


The following object is masked from 'package:stats':

    as.dendrogram
```

```
library(ggplot2)
library(MASS)
set.seed(2201)

# Define the cluster centers and sizes
centers <- matrix(c(25, 25, -25, -25, 25, -20, -20, 20), ncol = 2, byrow = TRUE)
sizes <- c(1000, 1000, 1000, 1000, 1000, 1000, 1000, 1000, 1000, 1000, 250, 100)

# Generate synthetic clusters with different characteristics
cluster1 <- mvrnorm(n = sizes[1], mu = centers[1,], Sigma = matrix(c(9, 2, 2, 9), ncol = 2))
cluster2 <- mvrnorm(n = sizes[2], mu = centers[2,], Sigma = matrix(c(12, 0.5, 0.5, 4), ncol =
cluster3 <- mvrnorm(n = sizes[3], mu = centers[3,], Sigma = matrix(c(6, 0.3, 0.3, 18), ncol =
cluster4 <- mvrnorm(n = sizes[4], mu = centers[4,], Sigma = matrix(c(15, 0.1, 0.1, 15), ncol

# Cluster 5: Sine curve distribution
x_values <- seq(-40, -20, length.out = 1000)
y_values <- sin(x_values) * 2 + rnorm(1000, mean = 0, sd = 0.2) # Adding some noise to the si
cluster5 <- cbind(x_values, y_values)

# Cluster 6 & 7: Intersecting clusters
cluster6 <- mvrnorm(n = 1000, mu = c(-5, -5), Sigma = matrix(c(7, 1.3, 1.3, 8), ncol = 2))
cluster7 <- mvrnorm(n = 1000, mu = c(-3, -7), Sigma = matrix(c(7, 1.3, 1.3, 5), ncol = 2))

# Cluster 8: Points along an arc
theta <- runif(1000, min = pi, max = 2 * pi)
r <- 5
```

```r
cluster8 <- cbind(r * cos(theta) + 5, r * sin(theta) + 15)

# Cluster 9: Points in a "moon" shape
theta <- runif(1000, min = 0, max = pi)
r <- 10 + runif(1000, min = -1, max = 1)
cluster9 <- cbind(r * cos(theta) + 30, r * sin(theta))

# Cluster 10: Points in a "ring" shape around all other clusters
theta <- runif(1000, min = 0, max = 2 * pi)
r_inner <- 48
r_outer <- 52
r <- sqrt(runif(1000, min = r_inner^2, max = r_outer^2))
cluster10 <- cbind(r * cos(theta), r * sin(theta))

# Noise and Outliers
noise <- cbind(runif(250, -60, 60), runif(250, -60, 60))
outliers <- cbind(rnorm(100, 0, 20), rnorm(100, 0, 20))

data_full <- rbind(cluster1, cluster2, cluster3, cluster4, cluster5,
                   cluster6, cluster7, cluster8, cluster9, cluster10, noise, outliers)
df_full <- as.data.frame(data_full)
colnames(df_full) <- c("X", "Y")
```

Execution and Comparison

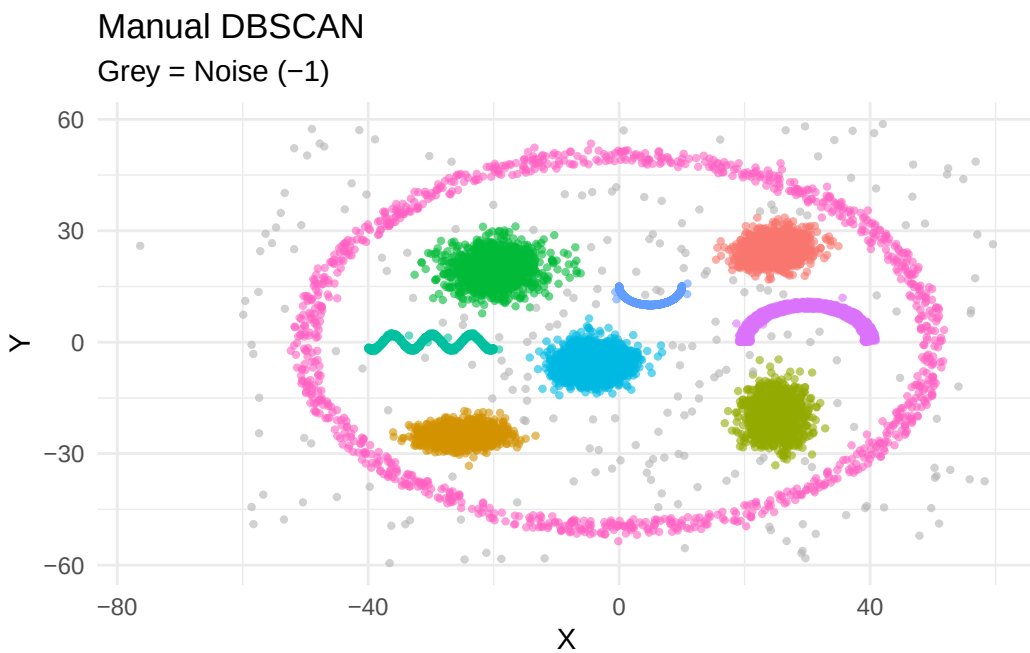
```r
set.seed(2201)
# Run Manual DBSCAN
results_dbscan <- manual_dbscan(df_full, eps = 2.5, minPts = 4)
df_full$Cluster <- as.factor(results_dbscan)

# Run K-means for Comparison
km_result <- kmeans(df_full, centers = 4, nstart = 25)
df_full$Kmeans_Cluster <- as.factor(km_result$cluster)
```

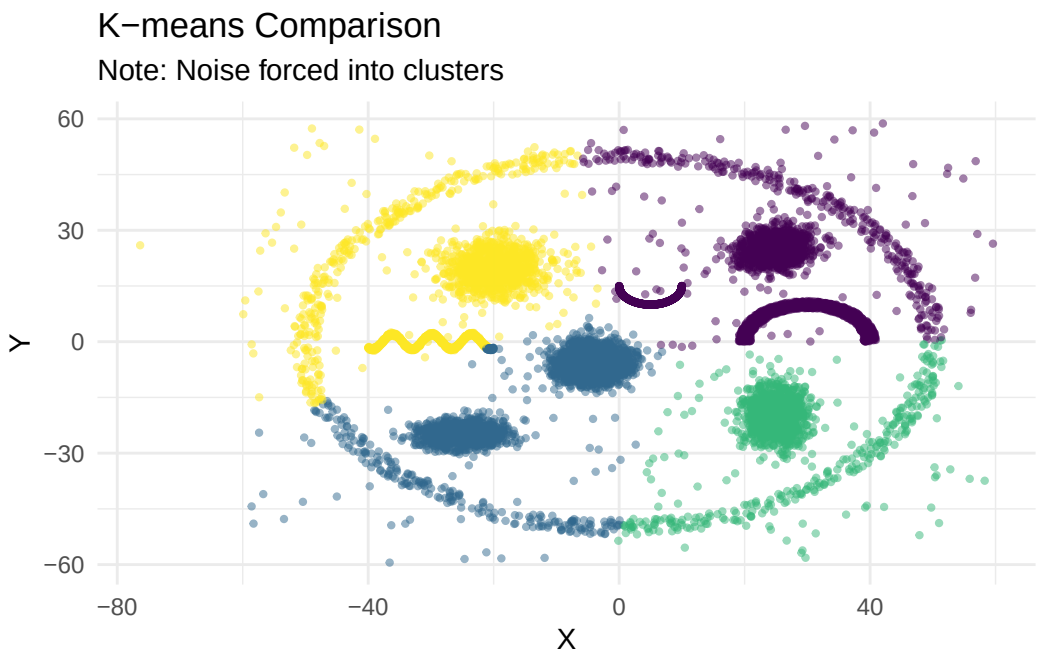
Visualization

```r
# DBSCAN Results
ggplot(df_full, aes(x = X, y = Y, color = Cluster)) +
```

```
  geom_point(alpha = 0.6, size = 0.8) +
  scale_color_manual(values = c("-1" = "grey70",
                                setNames(scales::hue_pal()(length(unique(results_dbscan))-1)
                                         unique(results_dbscan)[unique(results_dbscan) != -1
  theme_minimal() +
  labs(title = "Manual DBSCAN", subtitle = "Grey = Noise (-1)") +
  theme(legend.position = "none")
```




```
# K-means Results
ggplot(df_full, aes(x = X, y = Y, color = Kmeans_Cluster)) +
  geom_point(alpha = 0.5, size = 0.8) +
  theme_minimal() +
  scale_color_viridis_d() +
  labs(title = "K-means Comparison", subtitle = "Note: Noise forced into clusters") +
  theme(legend.position = "none")
```



Application of Algorithm